

Administración de Memoria

Módulo 8

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"

Módulo 8: Administración de Memoria

- Base
- Intercambio (Swapping)
- Alocación Contigua
- Paginado
- Estructura de la Tabla de Páginas
- Segmentación

Objetivos

- Describir distintas formas de organizar el hardware de memoria.
- Discutir varias técnicas de administración de memoria, incluyendo paginación y segmentación.

(este módulo debe resultar un repaso de la materia Arquitectura)

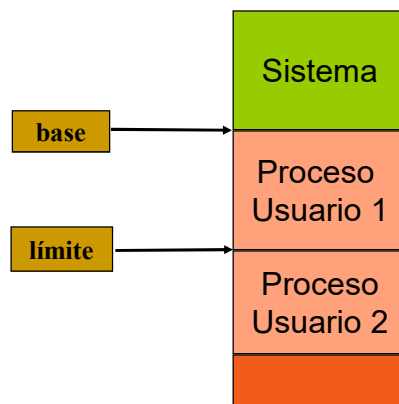
Base

- ▶ Un programa para su *ejecución*, debe ser traído del disco a la memoria y ubicado dentro de un proceso.
- ▶ La memoria principal y los registros son las únicas unidades de almacenamiento accedidas directamente por la CPU.
- ▶ El tiempo de acceso a
 - ▶ los registros de CPU, se hace en un pulso de reloj o menos.
 - ▶ la memoria principal puede tomar muchos ciclos.
- ▶ El **Caché** se utiliza para reducir la diferencia de velocidades. Se ubica entre la memoria principal y los registros de CPU.
- ▶ Para asegurar una correcta operación se requiere cierta protección de memoria.

Gestión de la memoria

- Dividir la memoria para acomodar múltiples procesos
- Debemos asegurar que c/ proceso disponga de un espacio de memoria separado.
- Esta protección se logra con dos registros (Base y Límite)
- el HW de la CPU compara todas las direcciones generadas en modo usuario con el contenido de estos registros
- Solo el SO podrá cargar los registros **base** y **límite** en modo kernel, permitiendo la carga de los programas de los usuarios en la memoria de usuario.
- El esquema evita que un programa de usuario modifique (accidentalmente o no) el código del SO o de otros usuarios. Cualquier intento genera un error fatal.

Espacio de dirección lógico

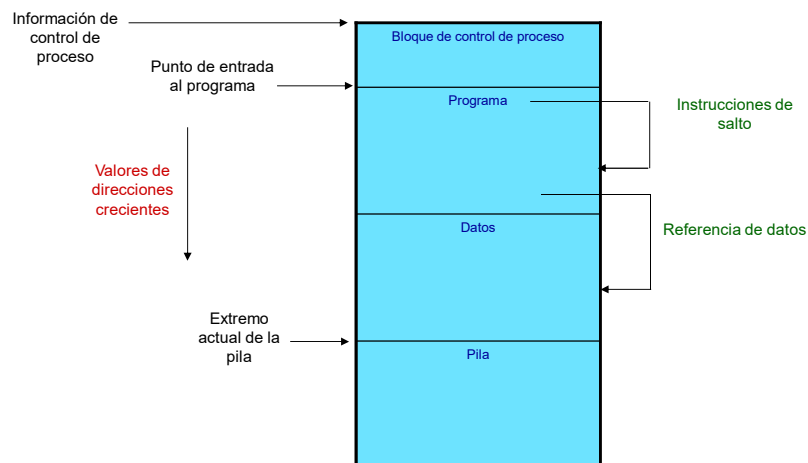


Requisitos de la gestión de la memoria

■ Reubicación

- El programador no sabe en qué parte de la memoria principal se situará el código del programa cuando se ejecute
- Mientras el programa se está ejecutando, éste puede llevarse al disco y traerse de nuevo a la memoria principal en un área diferente (reubicado)
- El HW del procesador y el SW del SO *deben traducir* las referencias de memoria del código del programa en direcciones de memoria físicas.

Requisitos de la gestión de la memoria



Requisitos de direccionamiento para un proceso

Requisitos de la gestión de la memoria

■ Protección

- Los procesos no deberían ser capaces de referenciar sin permiso posiciones de memoria principal de otro proceso
- Es **imposible** comprobar las direcciones absolutas en el tiempo de compilación
- Deben comprobarse en el tiempo de ejecución
- El procesador (hardware), es el que debe satisfacer el requisito de protección de memoria. El SO no puede anticipar todas las referencias de memoria que un programa hará

Requisitos de la gestión de la memoria

■ Compartición

- Permite a varios procesos acceder a la misma porción de memoria principal
- Es mejor permitir que cada proceso pueda acceder a la misma copia del programa en lugar de tener su propia copia separada.
- El sistema de gestión de memoria (MMU), debe permitir el acceso controlado a áreas de memoria compartida sin comprometer la protección.

Requisitos de la gestión de la memoria

■ Organización lógica

- Los programas están escritos en módulos
- Los módulos se pueden escribir y compilar independientemente
- Se pueden proporcionar diferentes grados de protección a los módulos (sólo lectura, sólo ejecución)
- Se pueden compartir módulos entre los procesos

Requisitos de la gestión de la memoria

■ Organización física

- La memoria principal disponible, podría ser insuficiente para un programa más sus datos
 - ▶ La técnica de superposición (*overlaying*) permite asignar la misma región de memoria a varios módulos
- El programador en tiempo de codificación, no conoce cuánto espacio estará disponible ó dónde se localizará dicho espacio.
- Por tanto la tarea de mover la información entre dos niveles de memoria (principal y secundaria) es responsabilidad del sistema y es la **esencia de la gestión de memoria**.

Mapeo de Instrucciones y Datos a Direcciones de Memoria

El mapeo puede ocurrir en tres etapas diferentes.

- **Tiempo de Compilación:** Si la locación de memoria es conocida a priori, puede ser generado código absoluto; si la dirección de comienzo cambia, debe recompilar.
- **Tiempo de Carga:** si la locación de memoria no es conocida en tiempo de compilación, debe generar código *reubicable*
- **Tiempo de Ejecución:** El mapeo es retrasado hasta el momento de corrida. El proceso puede ser movido durante su ejecución de un segmento de memoria a otro. Necesita soporte de hardware para el mapeo de las direcciones (p.e., registros *base - límite*).

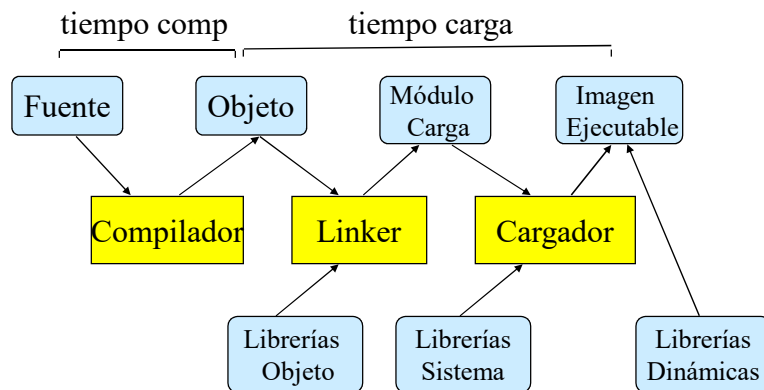
Carga Dinámica

- Las rutinas no son cargadas hasta que son invocadas.
- Mejor utilización del espacio de memoria; las rutinas no usadas no son cargadas.
- Es útil cuando hay que manejar grandes cantidades de código para casos poco frecuentes.
- No requiere un soporte especial del sistema operativo.

Enlace Dinámico

- El enlace es pospuesto hasta el tiempo de ejecución.
- Se usan pequeños fragmentos de código, *stub*, para localizar las rutinas apropiadas de la librería residente en memoria.
- El stub se sustituye a sí mismo con la dirección de la rutina y la ejecuta.
- El SO necesita verificar si la rutina está en las direcciones de memoria del proceso.
- Todos los procesos que utilicen una determinada librería, sólo necesitan ejecutar una copia del código de la librería.
- Similar a la carga dinámica, pero efectuando el enlace en tiempo de ejecución: bibliotecas dinámicas (DLL) o (shlib)

Mapeo de Direcciones



Espacio de Direcciones Lógico vs. Físico

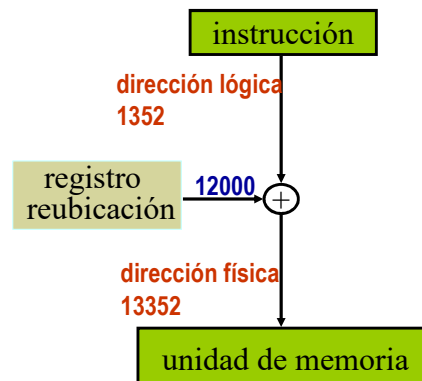
- El concepto de *espacio de direcciones lógico* está limitado a un espacio de *direcciones físicas* separado.
 - *Dirección Lógicas* – generadas por la CPU; también llamadas *direcciones virtuales*.
 - *Dirección Física* – dirección vista por la unidad de memoria.
- Las direcciones lógicas y físicas son las mismas en tiempo de compilación y en los esquemas de mapeo de direcciones en tiempo de carga;
- difieren en el esquema de mapeo de direcciones en tiempo de ejecución.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Unidad de Administración de Memoria (MMU)

- El MMU es el dispositivo hardware que mapea las direcciones virtuales a las físicas.
- En el esquema MMU, el valor del *registro de reubicación* es agregado a cada dirección generada por el proceso del usuario en el momento que es presentada a la memoria.
- Los programas de usuario ven direcciones lógicas, nunca ven direcciones físicas reales.



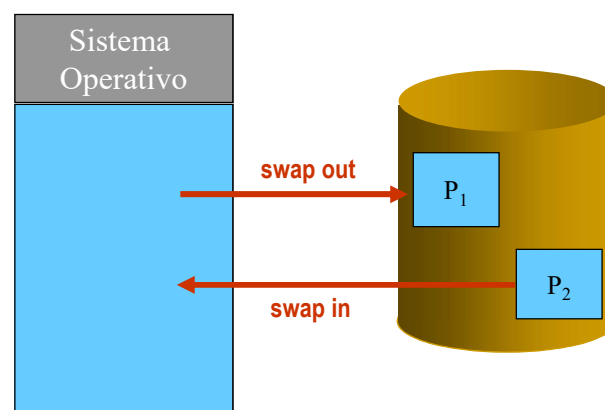
JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Intercambio (Swapping)

- Un proceso puede ser intercambiado (*swapped*) temporalmente fuera de la memoria a un almacenamiento de respaldo (*backing store*), y luego devuelto a la misma para continuar su ejecución.
- Backing store – espacio en disco lo suficientemente grande para acomodar copias de las imágenes de memoria de todos los usuarios, provee acceso directo a todas estas copias.
- *Roll out, roll in* – variante del intercambio usado en algoritmos de planificación basados en prioridades; procesos con baja prioridad son intercambiados con los de alta prioridad que pueden ser cargados y ejecutados.
- La mayor parte del tiempo de intercambio es tiempo de transferencia y es directamente proporcional a la cantidad de memoria intercambiada.

Visión Esquemática del Intercambio



Alocación Contigua

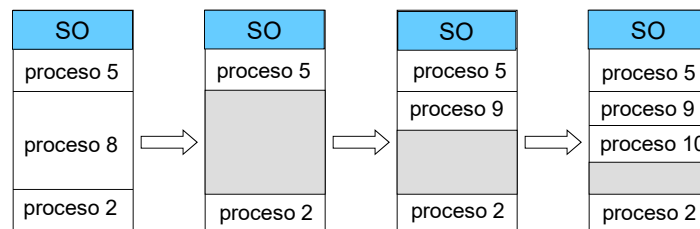
- La memoria principal se divide, en dos particiones:
 - Parte del SO residente, generalmente en memoria baja con el vector de interrupciones.
 - Los procesos de usuario se mantienen en la parte alta de la memoria.
- Alocación en partición simple
 - Usa un esquema de registro de reubicación para proteger tanto a los procesos de usuario, uno de otro, como al SO.
 - El registro de reubicación contiene el valor de la dirección física mas pequeña;
 - el registro límite contiene el rango de las direcciones lógicas – cada dirección lógica debe ser menor que el registro límite.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Alocación Contigua (Cont.)

- Alocación contigua en partición múltiple
 - Agujero – bloque de memoria disponible; de variados tamaños esparcidos por la memoria.
 - Cuando un proceso llega, es alocado en memoria en un agujero suficientemente grande para alojarlo.
 - El SO mantiene información sobre:
 - a) particiones alojadas b) particiones libre (agujero)



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Problema de Alocación Dinámica

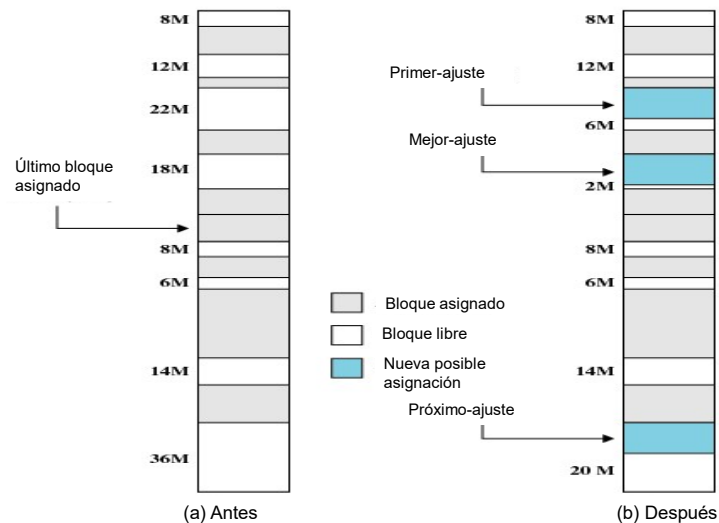
Como satisfacer un requerimiento de tamaño n de una lista de agujeros libres

- **Primer lugar:** Aloca en el *primer* agujero lo suficientemente grande.
- **Próximo lugar:** Aloca en el próximo lugar luego de haber usado el primero.
- **Mejor lugar:** Aloca en el agujero *mas chico* que lo puede alojar; debe buscar en la lista completa, salvo que sea ordenada por tamaño. Produce el agujero restante mas chico.
- **Peor lugar:** Aloca el agujero mas grande; debe buscar en la lista completa. Produce el agujero restante mas grande.

El *primer lugar* y el *mejor lugar* son mejores que el *peor lugar* en términos de velocidad y utilización del almacenamiento.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria



Ejemplo: Configuración de la memoria antes y después de la asignación de un **bloque de 16 Mbytes**

JRA © - LRS 2025

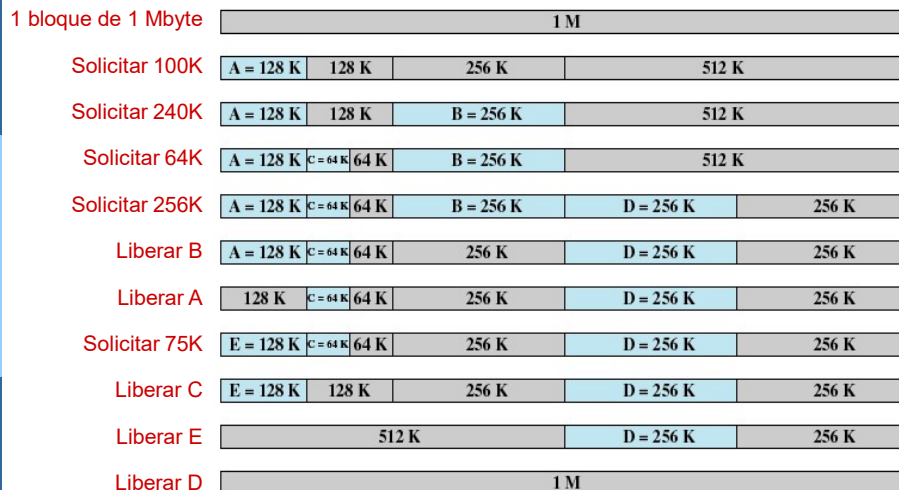
Sistemas Operativos – Administración de Memoria

Buddy System (Alocación)

Es un sistema de alocación de memoria utilizado por algunos SO.

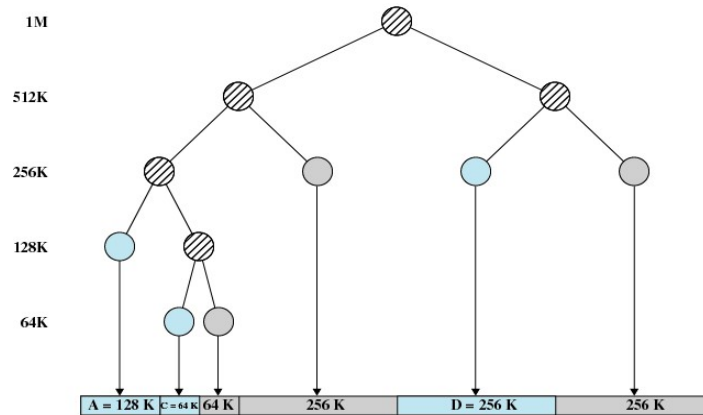
- El espacio completo disponible es tratado como un bloque de tamaño 2^U
- Si hay una solicitud de tamaño s , tal que $2^{U-1} < s \leq 2^U$, **se asigna el bloque completo**.
 - De otra manera el bloque es dividido en dos “compañeros” iguales.
 - El proceso continúa hasta que es generado el bloque mas pequeño mayor o igual que s .

Buddy System (Alocación)



Ejemplo de sistema Buddy

Buddy System (Alocación)



Representación en forma de árbol del sistema *Buddy*

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

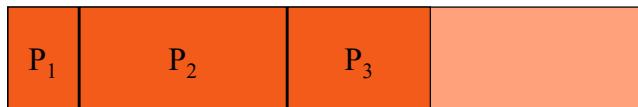
Fragmentación

- **Fragmentación Externa** – existe espacio de memoria suficiente para satisfacer una demanda, pero no es contiguo.
- **Fragmentación Interna** – la memoria ocupada puede ser ligeramente más grande que la demandada; esta diferencia de tamaño es memoria interna de la partición, pero no es usada.
- Se reduce la fragmentación externa por compactación
 - Mover el contenido de memoria de modo de dejar toda la memoria libre en un solo bloque.
 - es posible *solo* si la reubicación es **dinámica**, y es hecha en tiempo de ejecución.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

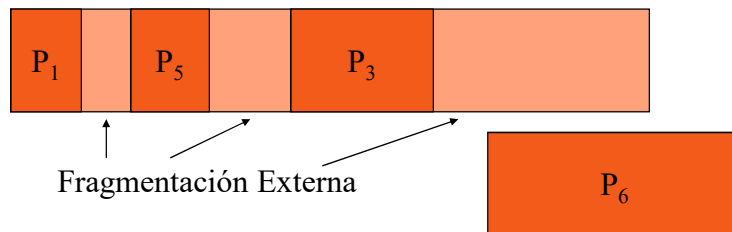
Fragmentación (Cont.)



Fragmentación (Cont.)



Fragmentación (Cont.)



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Paginado

- El espacio de direcciones puede no ser contiguo; el proceso es alojado en la memoria física donde haya lugar.
- La **memoria física** se divide en bloques de tamaño fijo llamados **cuadros** (el tamaño es potencia de 2, entre 512 bytes $\rightarrow 2^9$ y 16MB $\rightarrow 2^{24}$).
- La **memoria lógica** se divide en bloques de tamaño similar llamados **páginas**.
- El SO guarda información de todos los cuadros libres.
- Para correr un programa de n páginas, se necesita encontrar n cuadros libres y cargar el programa.
- Para traducir las direcciones lógicas a físicas, se establece una **tabla de páginas**.
- Fragmentación interna.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

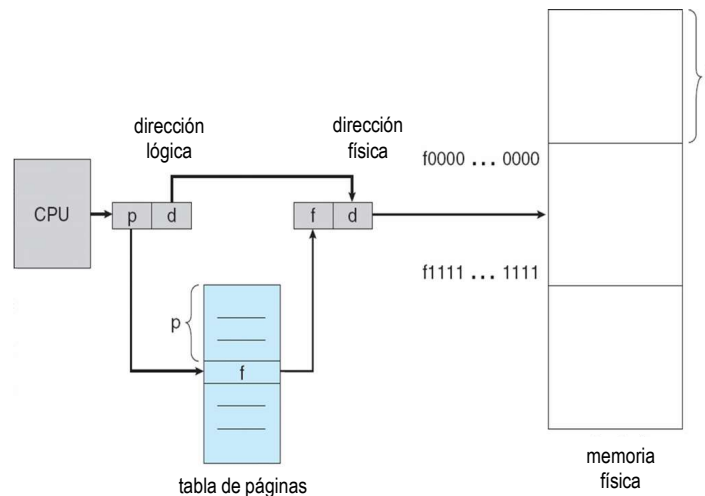
Esquema de Traducción de Direcciones

- La dirección generada por la CPU está dividida en:
 - **Número de página (p)** – usado como un índice en la *tabla de páginas* la cual contiene la dirección base de cada página en la memoria física.
 - **Desplazamiento en la página (d)** – combinado con la dirección base, permite definir la dirección física de memoria que es enviada a la unidad de memoria.

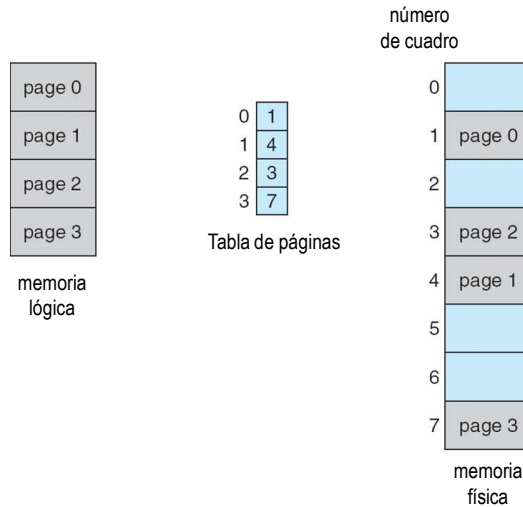
num pagina	desplazamiento
p	d
$m - n$	n

- Para un espacio de direcciones lógicas 2^m y tamaño de página 2^n

Hardware de Paginado



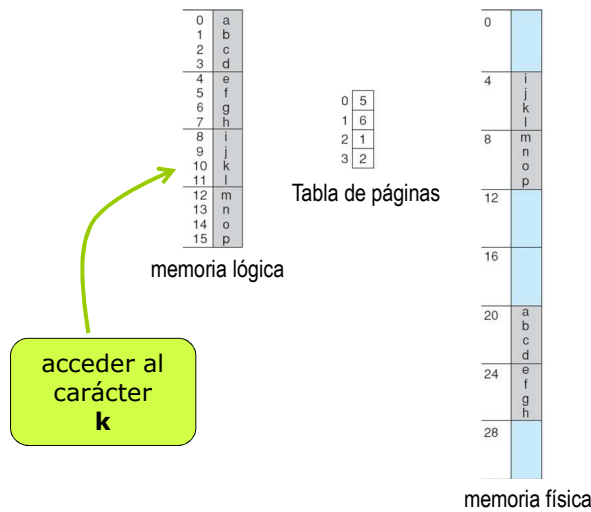
Modelo de Paginado de Memoria Lógica y Física



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Ejemplo de Paginado

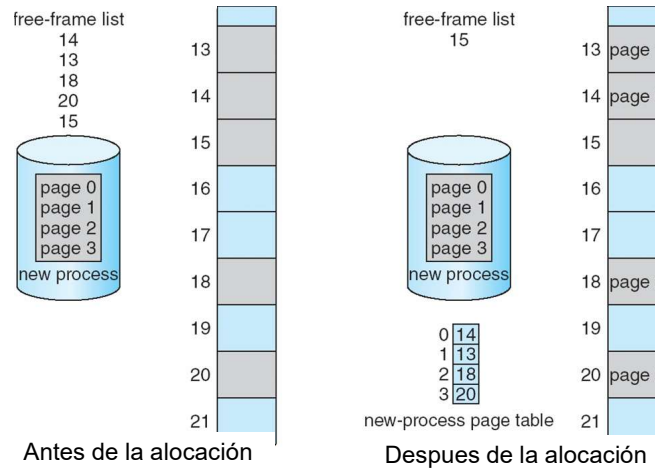


Memoria física de 32 bytes y páginas de 4bytes (8 páginas)

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Cuadros Libres



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Implementación de la Tabla de Páginas

- La tabla de páginas es guardada en memoria principal.
- El **registro base de tablas de páginas (PTBR)** apunta a la tabla de páginas.
- El **registro de longitud de la tabla de páginas (PTLR)** indica el tamaño de la tabla de páginas.
- En este esquema, cada acceso a instrucción/dato requiere dos accesos a memoria. Uno para la tabla de páginas y otro para el dato/instrucción.
- El problema al doble acceso puede ser resuelto usando un caché hardware especial de adelantamiento llamado **registro asociativo** o **TLBs (translation look-aside buffers)**

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

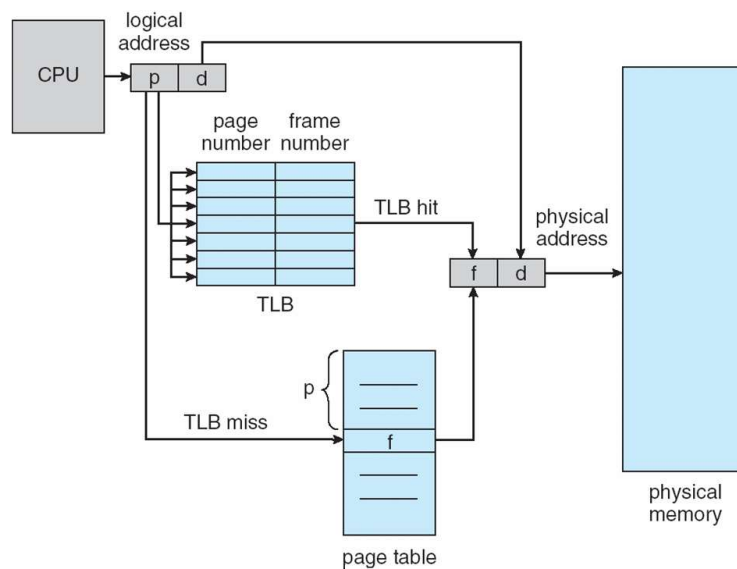
Registro Asociativo

- Registro Asociativo – búsqueda paralela

# página	# cuadro

- Traducción de dirección (**p**, **d**)
 - Si **p** está en un registro asociativo, tome el # cuadro.
 - Sino tome el # cuadro desde la tabla de páginas en memoria

Hardware de Paginado con TLB



Tiempo Efectivo de Acceso

- Búsqueda Asociativa = ε unidad de tiempo
- Suponga que el acceso memoria es de 1 microsegundo
- Tasa de acierto – porcentaje de veces que la página es encontrada en los registros asociativos; está relacionado con el número de registros asociativos.
- Tasa de acierto = α
- Tiempo Efectivo de Acceso (TEA)

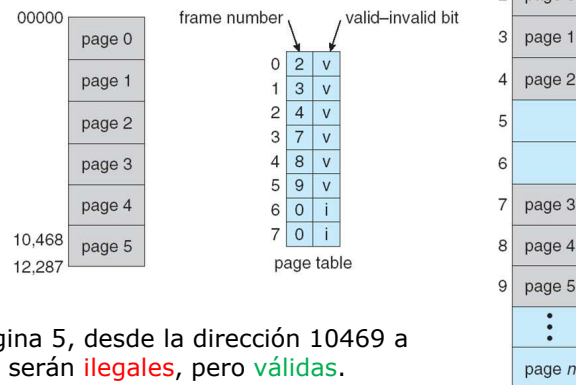
$$\begin{aligned}\text{TEA} &= (1 + \varepsilon) \alpha + (2 + \varepsilon)(1 - \alpha) \\ &= 2 + \varepsilon - \alpha\end{aligned}$$

Protección de Memoria

- La protección de memoria se implementa asociando un bit con cada cuadro.
- Un bit de *válido-Inválido* agregado a cada entrada en la tabla de páginas:
 - “**válido**” indica que la página asociada, está en el espacio de direcciones lógicas del proceso, por lo tanto es una página legal.
 - “**Inválido**” indica que la página no está en el espacio de direcciones lógicas del proceso.

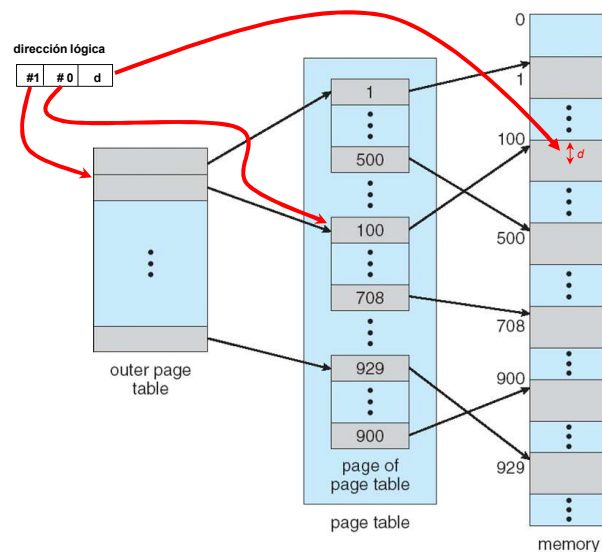
Bit de Válido (v) o Inválido (i) en una Tabla de Páginas

- ✓ Espacio de direcciones 14bits $\rightarrow 2^{14} = 16\text{Kb}$
- ✓ Tamaño del proceso 10.468 bytes
- ✓ Tamaño de página 2Kb



- ✓ En la pagina 5, desde la dirección 10469 a la 12287 serán **ilegales**, pero **válidas**.
- ✓ Desde 12288 hasta la 16384 son **inválidas**

Esquema de Tabla de Páginas de dos Niveles



Ejemplo de Paginado en dos Niveles

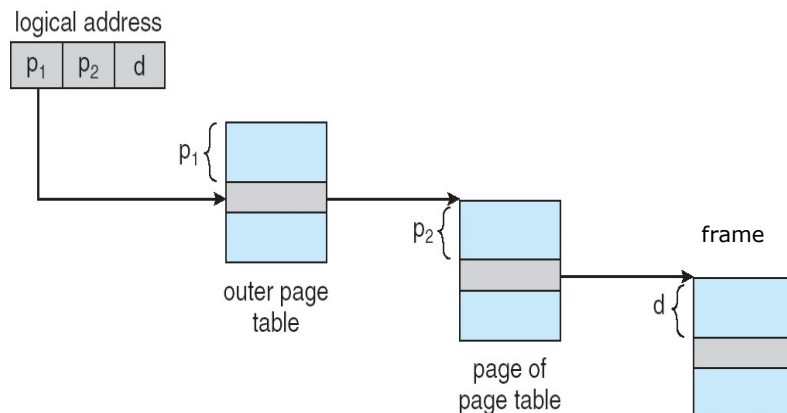
- Una dirección lógica (en una máquina de 32-bits con páginas de 4K) es dividida en:
 - Un número de página consistente de 20 bits.
 - Un desplazamiento en la página (offset) de 12 bits.
- Dado que la **tabla de páginas** está paginada, el número de página está a su vez dividido en:
 - Un número de página de 10 bits.
 - Un desplazamiento dentro de la página de 10 bits.
- La dirección lógica queda como sigue:

número de pag		desp en la pag
p_1	p_2	d
10	10	12

donde p_1 es un índice en la tabla de páginas exterior, y p_2 es el desplazamiento dentro de la página en la tabla de páginas interior.

Esquema de Traducción de Dirección

- Esquema de traducción de dirección para una arquitectura paginada de 32-bits en dos niveles



Esquema de Paginado en Tres Niveles

La CPU genera direcciones de 64bits. Con tamaño de pagina de 4k

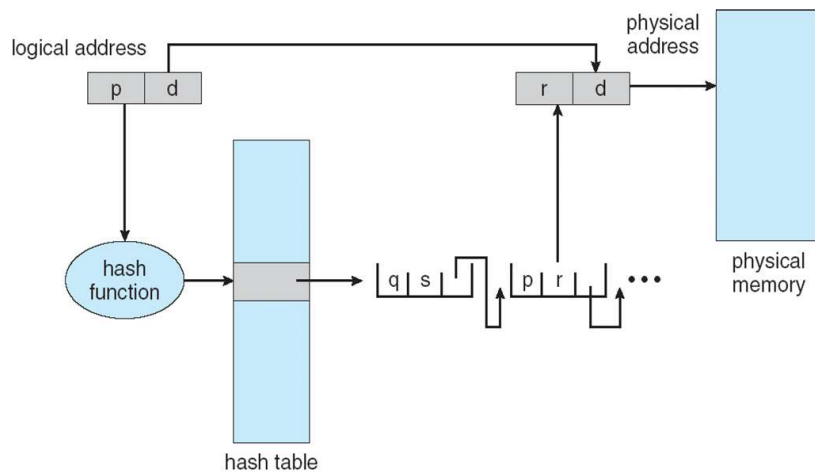
outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

Tablas de Páginas con Hash

- Común en espacio de direcciones > 32 bits
- Al número de página virtual se le aplica una función hash y se obtiene el índice a la tabla de página hash.
- Esta tabla contiene una lista enlazada de elementos que tienen como valor hash una misma ubicación.
- Los números de páginas virtuales son comparados con el campo1 de la lista correspondiente a esa entrada.
 - Si se encuentra, se extrae el cuadro físico (campo2).
 - Sino se explora el próximo elemento de la lista hasta encontrarla.

Tablas de Páginas con Hash



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

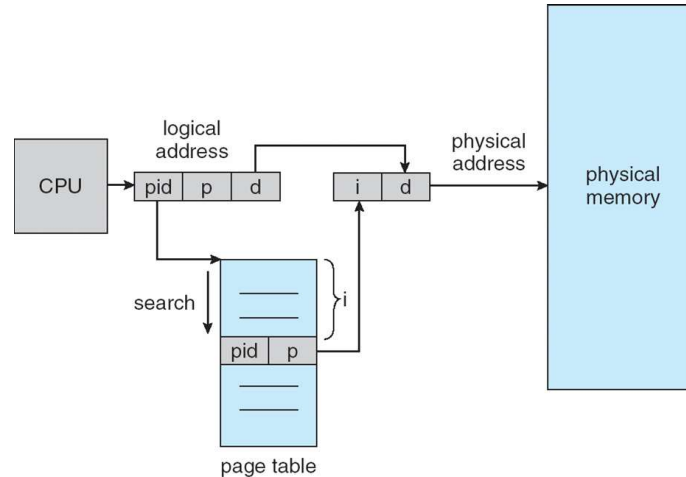
Tabla de Páginas Invertida

- Tiene una entrada por c/página real de memoria física (marco)
- Cada entrada consiste de la dirección virtual de la página almacenada en la locación real de memoria, con información del proceso que es dueño de esa página.
- En el sistema sólo habrá una única tabla de páginas.
- Se reduce el espacio de memoria necesaria para almacenar cada tabla de páginas, pero se incrementa el tiempo necesario para buscar en la tabla cuando ocurre una referencia a una página.
- Se usa una tabla hash para limitar la búsqueda a una (o a lo sumo unas pocas) entrada a la tabla de páginas.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Arquitectura de la Tabla de Página Invertida



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

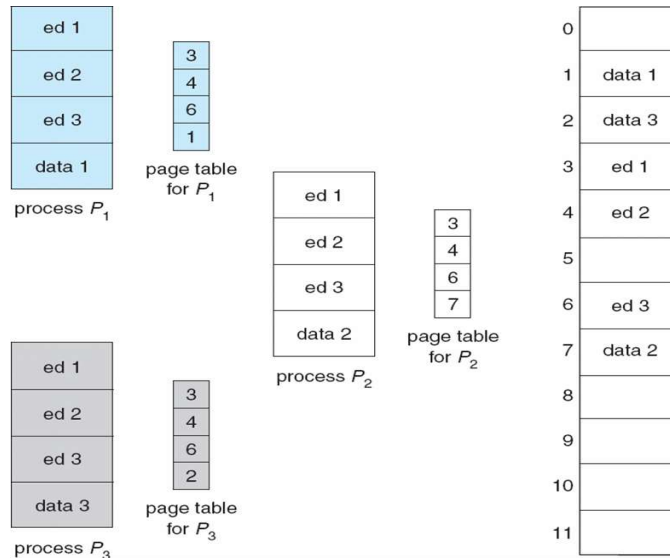
Páginas Compartidas

- Código compartido
 - Una copia de código, solamente de lectura (**reentrante**) compartido entre procesos (p.e., editores de texto, compiladores, sistemas de ventanas).
 - El código compartido debe aparecer en la misma locación en el espacio de direcciones de todos los procesos.
- Código privado y datos
 - Cada proceso guarda una copia separada de código y datos.
 - Las páginas del código privado y datos puede aparecer en cualquier lugar del espacio de direcciones lógico.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Ejemplo de Páginas Compartidas



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

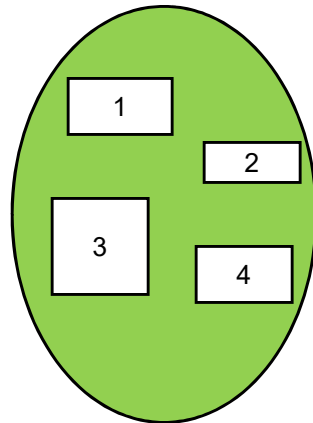
Segmentación

- Esquema de administración de memoria que soporta la visión que tiene el usuario de la memoria.
- Un programa es una colección de segmentos. Un segmento es una unidad lógica como:
 - ✓ programa principal,
 - ✓ procedimiento,
 - ✓ función,
 - ✓ variables locales, variables globales,
 - ✓ bloque común,
 - ✓ stack,
 - ✓ tabla de símbolos, arreglos

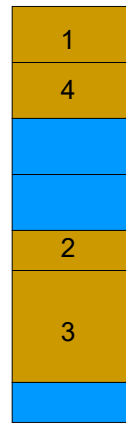
JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Visión Lógica de la Segmentación



Espacio de usuario



Espacio de memoria física

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Arquitectura de Segmentación

- Dirección lógica:
 $\langle \text{número-segmento, offset} \rangle$
- **Tabla de Segmentos** – mapean direcciones físicas en dos dimensiones; cada entrada en la tabla tiene:
 - **base** – contiene la dirección física inicial donde el segmento reside en memoria.
 - **límite** – especifica la longitud del segmento.
- **Registro base de la tabla de segmentos (STBR)** apunta a la locación de la tabla de segmentos en memoria.
- **Registro de longitud de la tabla de segmentos (STLR)** indica el número de segmentos usados por el programa;
el número de segmento **s** es legal si **s** < **STLR**.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

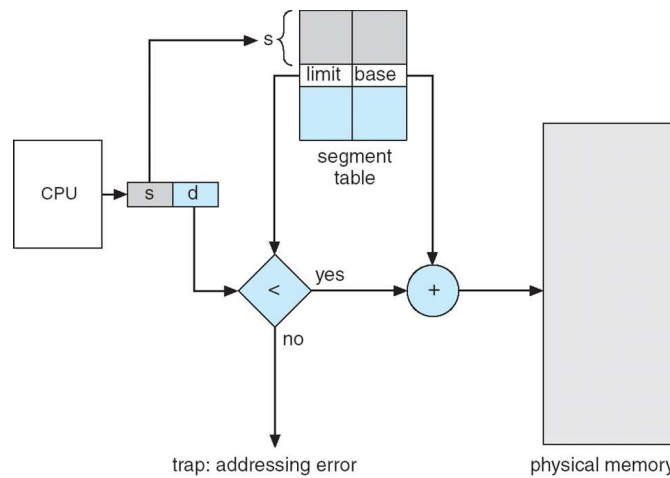
Arquitectura de Segmentación (Cont.)

- Relocación.
 - Dinámica → tamaños de los segmentos
 - por tabla de segmentos
- Compartir
 - segmentos compartidos
 - Igual número de segmento
- Alocación.
 - Primer lugar/mejor lugar
 - fragmentación externa

Arquitectura de Segmentación (Cont.)

- Protección. Con cada entrada en la tabla de segmentos se asocia:
 - bit de validación = 0 ⇒ segmento ilegal
 - privilegios **read/write/execute**
- Tanto los bits de protección asociados con segmentos como el código compartido ocurre a nivel de segmento.
- Dado que los segmentos varían en longitud, la alocación de memoria es un problema de alocación dinámica de almacenamiento.

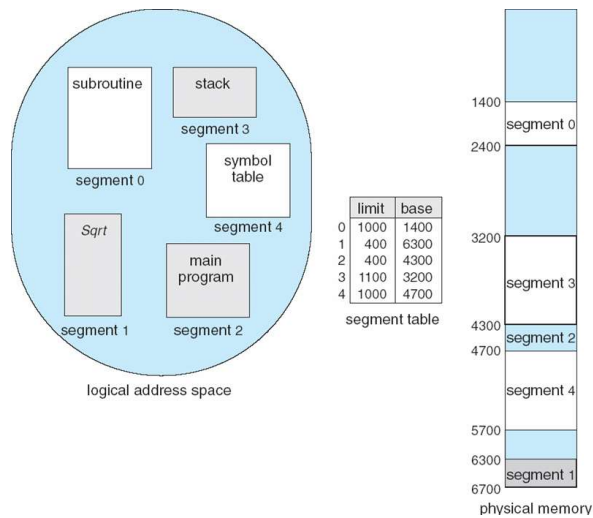
Hardware de Segmentación



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

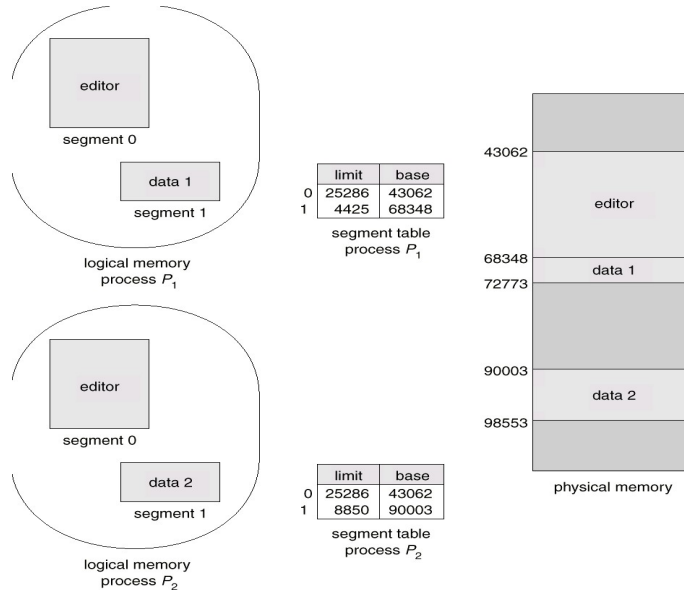
Ejemplo de Segmentación



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Segmentos Compartidos



JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Comparación de Estrategias de Manejo de Memoria

Asignación contigua, paginación, segmentación y la combinación de paginación-segmentación

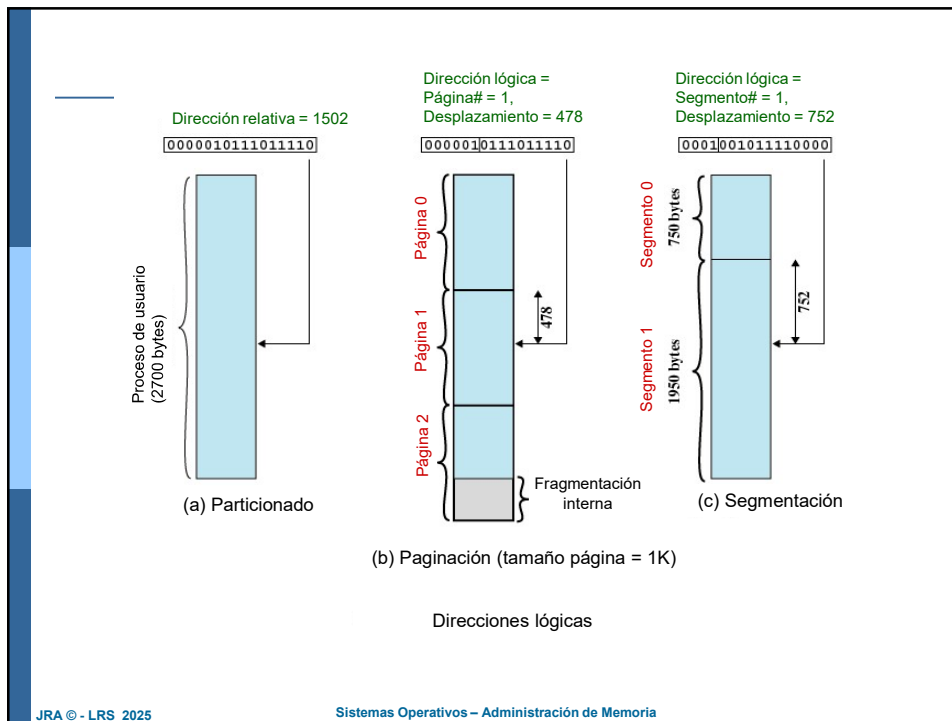
- **Soporte de Hardware** → con partición simple y múltiple resulta suficiente registro base o la pareja base-limite; para paginación y segmentación necesitan tablas de mapeo para definir las direcciones.
- **Rendimiento** → el algoritmo de gestión de memoria mas complejo, aumenta el tiempo de mapeo. Con particiones solo se necesita operación de comparación y suma; con paginación y segmentación podemos mejorar implementando TLBs.
- **Fragmentación** → Con asignación de tamaño fijo, partición simple y paginación: F. interna; con asignación de tamaño variable, partición múltiple y segmentación: F. externa.

JRA © - LRS 2025

Sistemas Operativos – Administración de Memoria

Comparación de Estrategias de Manejo de Memoria

- **Relocación** → la compactación soluciona la F. externa, necesita la reubicación dinámica de dir lógicas en t de ejecución. En tiempo de carga no es posible compactar.
- **Intercambio** (Swapping) → es posible añadir a cualquier algoritmo este mecanismo, permitiendo ejecutar mas procesos de los que cabrían en memoria.
- **Compartición** → incrementa el nivel de multiprogramación. Requiere mecanismos de paginación o segmentación que puedan compartir pequeños paquetes de info. Deben ser diseñados con sumo cuidado.
- **Protección** → Con paginación o segmentación distintas secciones de un programa se pueden declarar como de sólo lectura o lectura y escritura o de sólo ejecución. Necesarias para código o datos compartidos.



Fin

Módulo 8

Departamento de Informática
Facultad de Ingeniería
Universidad Nacional de la Patagonia "San Juan Bosco"